PLC Language — Factorial Processing Walkthrough

A Step-by-Step Trace Through the Compiler Pipeline

This document traces the complete execution of an iterative factorial program through every stage of the PLC language pipeline: lexical analysis, parsing, type checking, and tree-walking interpretation. The program computes `factorial(5) = 120`.

May 2026

Contents

1 The Source Program **2**

2 Stage 1 — Lexical Analysis **2**

2.1 Role of the Lexer 2

2.2 Token Stream Produced 3

2.3 Key Lexer Decisions 5

3 Stage 2 — Parsing **5**

3.1 Role of the Parser 5

3.2 AST Produced 5

3.3 How Each Construct is Parsed 6

3.3.1 Function Definition 6

3.3.2 While Statement 6

3.3.3 Assignment 7

3.3.4 Function Call 7

3.3.5 Expression Precedence 7

4 Stage 3 — Type Checking **7**

4.1 Role of the Type Checker 7

4.2 Two-Pass Function Registration 8

4.3 Checking the Top-Level Statements 8

4.3.1 Statement: `x = 5` 8

4.3.2 Statement: `f = factorial(x)` 8

4.3.3 Checking the Function Body 9

4.3.4 Statement: `print(f)` 9

4.4 Final Symbol Table 9

4.5 Why Recursion is Rejected 10

5 Stage 4 — Interpretation **10**

5.1 Role of the Interpreter 10

5.2 Interpreter Initialisation 10

5.3 Step-by-Step Execution 11

5.3.1 Step 1 — Register `factorial` 11

5.3.2 Step 2 — Evaluate `x = 5` 11

5.3.3 Step 3 — Evaluate `f = factorial(x)` 11

5.3.4 Step 4 — Bind `f` 12

5.3.5 Step 5 — Execute `print(f)` 12

6 Complete Execution Summary **13**

7 Key Language Design Decisions Illustrated **13**

1 The Source Program

The program used throughout this walkthrough is the iterative factorial implementation from `demos/demo_11_full.txt`. Recursion is not supported by the type checker, so factorial is computed using an accumulator variable inside a `while` loop.

```
def factorial(n) {
    result = 1;
    i = n;
    while (i != 0) {
        result = result * i;
        i = i - 1;
    }
    return result;
}

x = 5;
f = factorial(x);
print(f);
```

Listing 1: Iterative factorial program (demo_11_full.txt, lines 1–19)

The pipeline processes this source through four sequential stages, each described in its own section below.

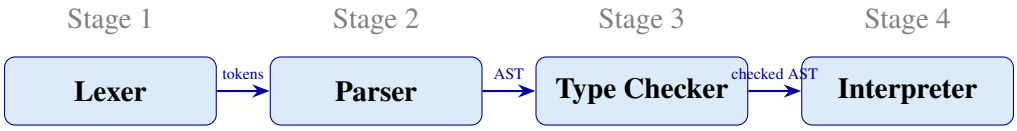

Figure 1: The four-stage PLC compiler pipeline.

2 Stage 1 — Lexical Analysis

2.1 Role of the Lexer

The lexer (`components/lexica.py`) scans the raw source text character by character and groups characters into *tokens*. Each token has a *type* (from the `TokenType` enum in `components/tokens.py`), a *lexeme* (the exact text matched), and a *line* and *column* position for error reporting. Whitespace and newlines are consumed but not emitted as tokens.

2.2 Token Stream Produced

Table 1 shows the complete token stream for the factorial program. Tokens are listed in the order the lexer emits them.

Table 1: Token stream for the factorial program.

Token type	Lexeme	Notes
DEF	def	Keyword
IDENTIFIER	factorial	Function name
LPAREN	(	
IDENTIFIER	n	Parameter name
RPAREN	)	
LBRACE	{	Begin function body
IDENTIFIER	result	
ASSIGN	=	
INT	1	Integer literal
SEMICOLON	;	
IDENTIFIER	i	
ASSIGN	=	
IDENTIFIER	n	
SEMICOLON	;	
WHILE	while	Keyword
LPAREN	(	
IDENTIFIER	i	
BANG_EQUAL	!=	Not-equal operator
INT	0	Integer literal
RPAREN	)	
LBRACE	{	Begin loop body
IDENTIFIER	result	
ASSIGN	=	
IDENTIFIER	result	
STAR	*	Integer multiply

Token type	Lexeme	Notes
IDENTIFIER	i	
SEMICOLON	;	
IDENTIFIER	i	
ASSIGN	=	
IDENTIFIER	i	
MINUS	-	Integer subtract
INT	1	Integer literal
SEMICOLON	;	
RBRACE	}	End loop body
RETURN	return	Keyword
IDENTIFIER	result	
SEMICOLON	;	
RBRACE	}	End function body
IDENTIFIER	x	
ASSIGN	=	
INT	5	Integer literal
SEMICOLON	;	
IDENTIFIER	f	
ASSIGN	=	
IDENTIFIER	factorial	Callee name
LPAREN	(	
IDENTIFIER	x	Argument
RPAREN	)	
SEMICOLON	;	
IDENTIFIER	print	Built-in function
LPAREN	(	
IDENTIFIER	f	
RPAREN	)	
SEMICOLON	;	
EOF	*(end)*	Signals end of input

2.3 Key Lexer Decisions

- **Keyword vs. identifier.** The lexer first reads a run of alphanumeric characters and underscores, then checks a keyword map. `def`, `while`, and `return` are emitted as their own keyword tokens; `factorial`, `n`, `result`, `i`, `x`, and `f` are emitted as `IDENTIFIER`.
- **Integer vs. float literals.** A sequence of digits without a decimal point is emitted as `INT`; a sequence containing a decimal point is emitted as `FLOAT`. No float literals appear in this program.
- **Float operators vs. integer operators.** The lexer distinguishes `*` (`STAR`) from `*.` (`STAR_DOT`) and `-` (`MINUS`) from `-.` (`MINUS_DOT`). Only integer operators appear here.
- **`!=` two-character token.** The lexer reads `!` and peeks ahead: if the next character is `=` it consumes both and emits `BANG_EQUAL`.

3 Stage 2 — Parsing

3.1 Role of the Parser

The parser (`components/parser.py`) is a hand-written recursive-descent parser. It consumes the token stream produced by the lexer and constructs an *Abstract Syntax Tree* (AST) whose nodes are dataclasses defined in `components/ast_nodes.py`. The root node is always a `Program`, which holds a flat list of top-level statements.

3.2 AST Produced

The parser produces the following tree for the factorial program. Indentation represents parent–child relationships.

```
Program
  FunctionDef name="factorial" params=["n"]
    Block
      Assign name="result"
        Literal value=1 (Integer)
      Assign name="i"
        Identifier name="n"
      While
        BinaryOp op="!="
          Identifier name="i"
          Literal value=0 (Integer)
```

```
      Block (loop body)
        Assign name="result"
          BinaryOp op="*"
            Identifier name="result"
            Identifier name="i"
        Assign name="i"
          BinaryOp op="-"
            Identifier name="i"
            Literal value=1 (Integer)
    Return
      Identifier name="result"
Assign name="x"
  Literal value=5 (Integer)
Assign name="f"
  FunctionCall name="factorial"
    Identifier name="x"
FunctionCall name="print"
  Identifier name="f"
```

Listing 2: Abstract Syntax Tree for the factorial program.

3.3 How Each Construct is Parsed

3.3.1 Function Definition

When the parser sees a `DEF` token it enters `_parse_func_def()`. It reads the function name, the comma-separated parameter list between parentheses, and then calls `_parse_block()` to consume the body enclosed in braces. The result is a `FunctionDef` node. No type annotations are written by the programmer; all type information is inferred later.

3.3.2 While Statement

On encountering `WHILE` the parser reads the condition expression between parentheses, then calls `_parse_block()` for the loop body. The condition `i != 0` is parsed as a comparison expression: first `i` is parsed as an `Identifier`, then `!=` is consumed, then `0` is parsed as an integer `Literal`, and together they form `BinaryOp(op="!=", left=Identifier("i"), right=Literal(0))`.

3.3.3 Assignment

A statement that starts with an `IDENTIFIER` followed by `ASSIGN` is parsed as an `Assign` node. For example, `result = result * i` becomes `Assign(name="result", value=BinaryOp("*", Identifier("result"), Identifier("i")))`.

3.3.4 Function Call

An `IDENTIFIER` immediately followed by `LPAREN` is parsed as a `FunctionCall`. The argument list between the parentheses is parsed as a comma-separated sequence of expressions. For `factorial(x)`, the single argument `x` is parsed as `Identifier("x")`.

3.3.5 Expression Precedence

The grammar implements standard arithmetic precedence through three levels of recursive-descent rules:

Rule	Handles
`_expr()`	Comparison (==, !=)
`_additive()`	Addition and subtraction (+, -, +., -.)
`_term()`	Multiplication and division (*, /, *., /.)
`_factor()`	Literals, identifiers, calls, grouped expressions

4 Stage 3 — Type Checking

4.1 Role of the Type Checker

The type checker (`components/type_checker.py`) performs a single-pass, syntax-directed traversal of the AST. Its goals are:

1. Build a *symbol table* mapping every variable and function name to its inferred type.
2. Verify that every operator is applied to operands of the correct type.
3. Verify that `while` and `if` conditions are `Boolean`.
4. Infer the parameter and return types of user-defined functions from their usage context.

5. Reject recursive function calls.

No type annotations are required from the programmer. Types are inferred entirely from context.

4.2 Two-Pass Function Registration

Before checking any expression or statement, the type checker makes a preliminary scan of the top-level statement list. Any `FunctionDef` node is registered in the symbol table as a `FunctionSymbol` with *placeholder* (unknown) parameter types. This ensures that a function can be referenced before its call site is seen.

4.3 Checking the Top-Level Statements

4.3.1 Statement: `x = 5`

The right-hand side is `Literal(5)`. The literal's value is the Python integer `5`, so its type is `Integer`. The type checker records `x : Integer` in the global symbol table.

4.3.2 Statement: `f = factorial(x)`

The right-hand side is `FunctionCall("factorial", [Identifier("x")])`. The type checker:

1. Looks up `x` in the symbol table: `Integer`.
2. This is the *first call* to `factorial`, so the placeholder parameter type for `n` is fixed to `Integer`.
3. The function body is now type-checked with `n : Integer` in scope (inside a dedicated function scope). The name `factorial` is pushed onto `_active_calls` to detect recursion.
4. The body checking proceeds (detailed in the next subsection).
5. The inferred return type of `factorial` is `Integer`.
6. `f : Integer` is recorded in the global symbol table.

4.3.3 Checking the Function Body

With `n : Integer` in the function scope:

Statement	Inferred type	Action
`result = 1`	`Integer`	Record `result : Integer` in function scope.
`i = n`	`Integer`	Look up `n`: `Integer`. Record `i : Integer`.
`while (i != 0)` condition	`Boolean`	Both operands `i` and `0` are `Integer`; `!=` on integers yields `Boolean`. Condition is `Boolean`: pass.
`result = result * i`	`Integer`	Both `result` and `i` are `Integer`; `*` on integers yields `Integer`; type of `result` remains `Integer`.
`i = i - 1`	`Integer`	Both `i` and `1` are `Integer`; `-` yields `Integer`.
`return result`	`Integer`	Sets function return type to `Integer`.

4.3.4 Statement: `print(f)`

`print` is a built-in. The type checker verifies exactly one argument is supplied and that the argument type is a valid language type. `f : Integer`: accepted.

4.4 Final Symbol Table

After type checking completes the global and function scopes contain:

Scope	Name	Kind	Type
Global	`factorial`	Function	params: [`Integer`], return: `Integer`
Global	`x`	Variable	`Integer`
Global	`f`	Variable	`Integer`
Function `factorial`	`n`	Variable	`Integer`
Function `factorial`	`result`	Variable	`Integer`
Function `factorial`	`i`	Variable	`Integer`

4.5 Why Recursion is Rejected

The type checker maintains a list `_active_calls`. When it begins type-checking the body of a function `f`, it appends `f` to that list. If a `FunctionCall` node for `f` is encountered while `f` is still in `_active_calls`, a `TypeCheckError` is raised. This makes a direct recursive factorial definition — such as `return n * factorial(n - 1)` — a compile-time error in this language. The iterative formulation avoids recursion entirely and compiles without error.

5 Stage 4 — Interpretation

5.1 Role of the Interpreter

The tree-walking interpreter (`components/interpreter.py`) traverses the type-checked AST and executes each node directly. It maintains a chain of `_Environment` objects (linked hash maps) to store variable bindings. Each function call creates a new environment whose *parent* is the call site's environment, providing lexically scoped name lookup.

5.2 Interpreter Initialisation

At the start of `execute()`, the interpreter makes a first pass over all top-level statements and registers every `FunctionDef` in an internal dictionary `_functions`. A global `_Environment` is then created for the top-level bindings. After registration, a second pass executes the non-`FunctionDef` statements in order.

5.3 Step-by-Step Execution

5.3.1 Step 1 — Register `factorial`

The `FunctionDef` node is stored in `_functions` under the key `"factorial"`. No code is executed yet; only the AST node is saved for later use.

$$\_functions = \{\ \texttt{"factorial"} \mapsto \mathit{FunctionRuntime}(\text{node})\ \}$$

5.3.2 Step 2 — Evaluate `x = 5`

The right-hand side `Literal(5)` evaluates to the Python integer `5`. The global environment is updated:

$$global\_env = \{\ \texttt{x} \mapsto 5\ \}$$

5.3.3 Step 3 — Evaluate `f = factorial(x)`

The right-hand side is a `FunctionCall`. The interpreter:

1. Evaluates the argument `x` in the current (global) environment: `5`.
2. Creates a new `call_environment` with `parent = global_env`.
3. Binds the parameter: `call_environment["n"] = 5`.
4. Executes the function body block in `call_environment`.

Inside the function body. **Assign `result = 1`.** `Literal(1)` evaluates to `1`.

$$call\_env = \{\ \texttt{n} \mapsto 5,\ \texttt{result} \mapsto 1\ \}$$

Assign `i = n`. Looking up `n` in `call_env` returns `5`.

$$call\_env = \{\ \texttt{n} \mapsto 5,\ \texttt{result} \mapsto 1,\ \texttt{i} \mapsto 5\ \}$$

While loop. Each iteration creates a fresh *child* environment linked to `call_env`, so the loop-body assignments write back through the parent chain. The following table traces each iteration.

Iteration	`i` **(before)**	**Condition** `i != 0`	`result = result * i`	`i = i - 1`	`i` **(after)**
1	5	`true`	$1 \times 5 = 5$	$5 - 1 = 4$	4
2	4	`true`	$5 \times 4 = 20$	$4 - 1 = 3$	3
3	3	`true`	$20 \times 3 = 60$	$3 - 1 = 2$	2
4	2	`true`	$60 \times 2 = 120$	$2 - 1 = 1$	1
5	1	`true`	$120 \times 1 = 120$	$1 - 1 = 0$	0
6	0	`false`	*loop exits*		

After the loop, `call_env` holds:

$$\textit{call\_env} = \{\, \texttt{n} \mapsto 5,\ \texttt{result} \mapsto 120,\ \texttt{i} \mapsto 0 \,\}$$

Return statement. `Return(Identifier("result"))` evaluates `result` to `120` and raises a `_ReturnSignal(120)`. The interpreter catches this signal in `_call_function()` and returns the value `120` as the result of the call.

5.3.4 Step 4 — Bind `f`

The function call returned `120`. The assignment stores it:

$$\textit{global\_env} = \{\, \texttt{x} \mapsto 5,\ \texttt{f} \mapsto 120 \,\}$$

5.3.5 Step 5 — Execute `print(f)`

The interpreter detects the built-in name `"print"`. It evaluates the argument `f` to `120`, determines its runtime type (`Integer`), formats the output string, and calls the output callback:

```
120 : Integer
```

This string is appended to the pipeline's output list and displayed to the user.

6 Complete Execution Summary

Table 2 provides a consolidated view of the entire pipeline from source text to final output.

Table 2: End-to-end pipeline summary for `factorial(5)`.

Stage	Result
Lexer	51 tokens emitted, including `DEF`, `WHILE`, `RETURN`, `BANG_EQUAL`, `STAR`, `MINUS`, several `INT` literals, and multiple `IDENTIFIER` tokens.
Parser	A `Program` AST with one top-level `FunctionDef` (`factorial`), two top-level `Assign` nodes (`x`, `f`), and one top-level `FunctionCall` (`print`).
Type Checker	All six symbols (`factorial`, `x`, `f`, `n`, `result`, `i`) inferred as `Integer`. No type errors.
Interpreter	`factorial(5)` computed iteratively over 5 loop iterations: $1\times5\times4\times3\times2\times1=120$. Global environment ends as $\{\texttt{x}\mapsto5,\ \texttt{f}\mapsto120\}$.
Output	`120 : Integer`

7 Key Language Design Decisions Illustrated

This walkthrough highlights several deliberate design choices:

1. **No explicit type declarations.** The programmer writes `result = 1` without stating that `result` is an integer. The type checker infers this from the literal `1`. Subsequent uses of `result` in `result * i` are then checked against the inferred `Integer` type.

2. **Separate integer and float operators.** The multiplication `result * i` uses `*`, not `*.`. If a `Float` variable were mistakenly used here, the type checker would raise an error, because `*` only accepts `Integer` operands.

3. **No implicit numeric coercion.** Every value in this program is an `Integer` throughout. There is no silent widening to `Float`.

4. **Recursion prohibited at type-check time.** A natural recursive definition `return n * factorial(n - 1)` would fail during Stage 3 with a `TypeCheckError: Recursive`

`function calls are not supported: factorial`. The iterative formulation using an accumulator is required.

5. **Return via exception signal.** The interpreter implements `return` by raising a private `_ReturnSignal` exception that carries the return value. This unwinds the call stack cleanly, even from inside nested blocks or `while` loops.

6. **Scoped environments.** Each `while` loop iteration and each function call runs in a new `_Environment` linked to its parent. Assignments inside the loop body propagate outward through the parent chain, so `result` and `i` are correctly updated in `call_env` on every iteration.